

Tarefas de Validação - Martinson

Para comprovar que você compreende seu código `klotski_opt.c`, execute uma das 3 tarefas abaixo. Estas tarefas validam seu entendimento do algoritmo.

Tarefa 1: Estatísticas de Busca com `--stats`

Descrição

Seu `solve()` já conta `count` (nós explorados). Adicione um flag `--stats` para imprimir um relatório detalhado ao final da busca.

Instruções

1. Modifique `main()` para aceitar: `programa entrada.txt --stats`
2. Passe uma flag `int verbose_stats` para `solve()`.
3. Declare variáveis globais para rastrear:
 - `int nos_explorados` - incrementado a cada `heap_pop()` (já existe como `count`)
 - `int nos_gerados` - incrementado cada vez que `heap_push()` é chamado
 - `int profundidade_solucao` - armazenar `current->g` quando solução for encontrada
 - `clock_t tempo_inicio/fim` - para medir tempo total
4. Ao encontrar solução (ou ao fim da busca se `--stats`), imprima:

```
===== ESTATÍSTICAS A* =====
Nós explorados: 54321
Nós gerados: 123456
Profundidade: 28
Tempo: 0.532 segundos
Razão gerado/explorado: 2.29
=====
```
5. Se **nenhuma solução** for encontrada, imprima as mesmas estatísticas sem “Profundidade”.

Critério de sucesso

- Flag `--stats` é reconhecido
 - Números são precisos (devem bater com a busca)
 - Tempo é medido em segundos
 - Saída é formatada claramente
-

Tarefa 2: Modo `--compact` para Solução Minimalista

Descrição

Atualmente, `reconstruct_path()` imprime cada movimento como peça direção (ex: X U). Implemente um modo `--compact` que: - Agrupa movimentos **sucessivos da mesma peça** em um único passo - Mostra o número total de células que a peça se moveu

Instruções

1. Modifique `main()` para aceitar: `programa entrada.txt --compact`
2. Ao invés de imprimir a cada movimento durante recursão, **armazene todos os movimentos** em um vetor:

```
typedef struct {
    char piece;
    char direction;
} Move;

Move solucao[MAX_MOVIMENTOS];
int num_movimentos = 0;
```
3. Após encontrar a solução, **processe o vetor** de movimentos e agrupe consecutivos:
 - Se `solucao[i].piece == solucao[i+1].piece` e `solucao[i].direction == solucao[i+1].direction`, incremente contador de células
 - Imprima: Peça X cima (3 células) ao invés de 3 linhas separadas
4. Exemplo de saída:

```
Passo 1: X cima (2 células)
Passo 2: A direita (1 célula)
Passo 3: X esquerda (3 células)
Total: 3 passos agrupados
```
5. Converta nomes de direções para português:
 - U → “cima”
 - D → “baixo”
 - L → “esquerda”
 - R → “direita”

Critério de sucesso

- Flag `--compact` é reconhecido
 - Movimentos são agrupados corretamente
 - Nomes de direção em português
 - Contagem de passos está correta
 - Totalizador no fim
-

Tarefa 3: Validação Robusta de Entrada

Descrição

Sua função `parse_input()` lê o arquivo, mas falta **validação defensiva**. Adicione verificações para garantir entrada bem-formada.

Instruções

- Após `parse_input()` retornar em `main()`, **antes de chamar** `solve()`, implemente validações:
 - Linhas com largura consistente:** todas as linhas têm o mesmo número de colunas
`// Dentro de parse_input: verificar que cada linha lida tem strlen == initial->cols`
 - Peça alvo presente e em local válido:** se um `TARGET_ROW/TARGET_COL` foi detectado na seção “# solução”, verifique que:
 - `TARGET_ROW` e `TARGET_COL` estão dentro dos limites do tabuleiro
 - A célula naquele local na matriz inicial está acessível (vazia `.` ou terá a peça alvo)
 - Ao menos 1 célula vazia:** há pelo menos um `.` no tabuleiro inicial
 - Peças identificadas corretamente:** cada peça tem altura e largura ≥ 1
 - Tabuleiro não está vazio:** há pelo menos 1 peça identificada
- Se qualquer validação falhar, imprima mensagem descritiva e **aborte** `main()` com `return 1`.
- Se todas as validações passarem, imprima:
Validação de entrada: ✓ OK

Critério de sucesso

- Código compila sem erros ou warnings novos
- Todas as 5 validações funcionam
- Entrada inválida é recusada com mensagem clara
- Entrada válida prossegue normalmente para busca
- Programa retorna código de saída apropriado (0 ou 1)

Resumo

Tarefa	Conceito-chave	O que você prova
1	Rastreamento de busca A*, heurística, performance	Que você entende ciclo explorado/gerado e timing
2	Armazenamento de traço, agregação de movimentos, tradução	Que você domina <code>reconstruct_path()</code> e pode processar a sequência
3	Validação defensiva, detecção de erros,	Que você entende quais estados são

Tarefa **Conceito-chave**
robustez

O que você prova
válidos no Klotski

Próximas Etapas

Escolha **uma tarefa** para implementar ao vivo durante a avaliação prática. Explique:

- Onde você inseriu o código (função principal, auxiliares, etc.)
- Quais variáveis/contadores criou
- Como testou com o arquivo de entrada disponível

Dica: Antes de começar:

- Revise `heap_push()` e `heap_pop()` para entender como `nos_gerados` e `nos_explorados` se relacionam
- Estude `reconstruct_path()` e pense como armazenar movimentos ao invés de imprimir
- Teste suas validações com entrada inválida (linhas diferentes, sem vazios, etc.)

Boa sorte! 🚀